

Orthogonal Range Queries

1 1-D Range Searching

- range query and split node
- algorithm formulation and cost analysis

2 Kd Trees

- two dimensional range queries
- building a Kd tree

3 Spatial Search in CGAL

- package wrappers available for Python

MCS 481 Lecture 13
Computational Geometry
Jan Verschelde, 23 September 2026

Orthogonal Range Queries

1 1-D Range Searching

- range query and split node
- algorithm formulation and cost analysis

2 Kd Trees

- two dimensional range queries
- building a Kd tree

3 Spatial Search in CGAL

- package wrappers available for Python

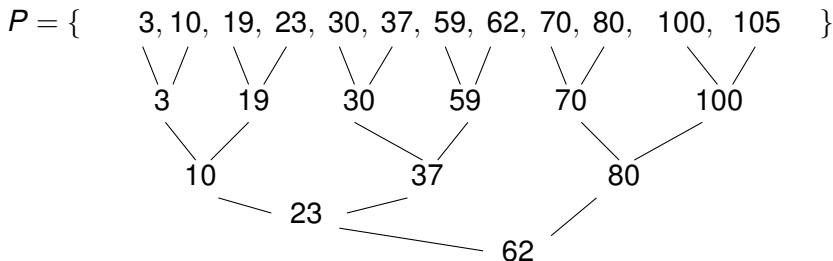
database queries

An example of a database query: list all employees within a given salary range and a number of children between 2 and 5.

Problem statement:

Input: $P = \{ p_1, p_2, \dots, p_n \}$ in balanced binary search tree,
a query range $[x : x']$.

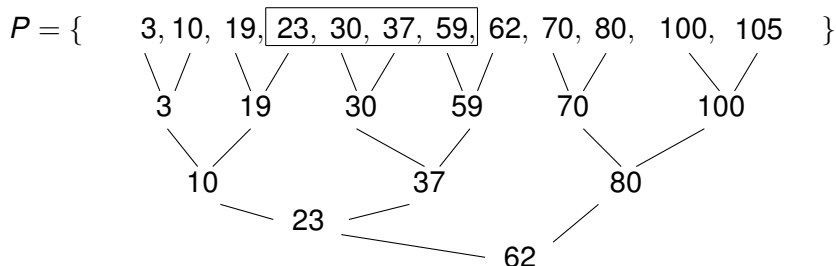
Output: $\{ y \in P \mid x \leq y \leq x' \}$.



an example of a range query

Input: $P = \{ 3, 10, 19, 23, 30, 37, 59, 62, 70, 80, 100, 105 \}$,
 $[x : x'] = [20 : 60]$.

Observe the output in the balanced binary search tree:

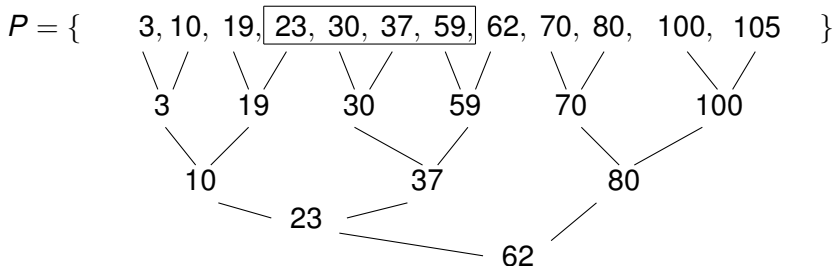


All output points are at the left of 62.

- report all points to the right of the nodes to the left of 23,
- report all points to the left of the nodes to the right of 23.

a split node

For $[x : x'] = [20 : 60]$, observe: $20 \leq 23 \leq 60$.



Definition (a split node)

For the range query $[x : x']$, the node v in a balanced binary search tree is a **split node** if its value x_v satisfies $x_v \geq x$ and $x_v < x'$.

finding a split node

The value x_v of a split node v satisfies $x_v \geq x$ and $x_v < x'$.

Algorithm FINDSPLITNODE($T, [x : x']$)

Input: T , a tree; and $[x : x']$, a range.

Output: a split node v .

- 1 $v = \text{ROOT}(T)$
- 2 while not ISLEAF(v) and ($x_v \geq x'$ or $x_v < x$) do
- 3 if $x' \leq x_v$
- 4 then $v = \text{LEFTCHILD}(v)$
- 5 else $v = \text{RIGHTCHILD}(v)$
- 6 return v

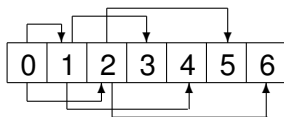
reporting all leaves of a tree

The subroutine `REPORTSUBTREE( $v$ )`

- traverses the tree rooted at v ; and
- reports the points stored at the leaves of this tree.

The amount of time is proportional to the number of points reported.

Exercise 1: Consider the array representation of a complete binary tree of $n = 2^d - 1$ elements. For a node at position p : left child is at $2p + 1$, right child is at $2p + 2$; and the parent of node at p is at $(p - 1)/2$.



Define the `ReportSubTree` for a tree stored in an array.

Orthogonal Range Queries

1 1-D Range Searching

- range query and split node
- algorithm formulation and cost analysis

2 Kd Trees

- two dimensional range queries
- building a Kd tree

3 Spatial Search in CGAL

- package wrappers available for Python

the algorithm for 1D range query

Algorithm 1DRANGEQUERY($T, [x : x']$)

- 1 $v_{\text{split}} = \text{FINDSPLITNODE}(T, [x : x'])$
- 2 If ISLEAF(v_{split}) then
- 3 check if point stored at v_{split} must be reported
- else
- 4 $v = \text{LEFTCHILD}(v_{\text{split}})$
- 5 while not ISLEAF(v) do
- 6 if $x \leq x_v$ then
- 7 REPORTSUBTREE(RIGHTCHILD(v))
- 8 $v = \text{LEFTCHILD}(v)$
- else
- 9 $v = \text{RIGHTCHILD}(v)$
- 10 check if point stored at v must be reported

1DRANGEQUERY continued

Now we follow the path starting at v to x' :

- 11 $v = \text{RIGHTCHILD}(v_{\text{split}})$
- 12 while not $\text{ISLEAF}(v)$ do
- 13 if $x_v < x'$ then
- 14 $\text{REPORTSUBTREE}(\text{LEFTCHILD}(v))$
- 15 $v = \text{RIGHTCHILD}(v)$
- else
- 16 $v = \text{LEFTCHILD}(v)$

Lemma (correctness of 1DRANGEQUERY)

Algorithm 1DRANGEQUERY is correct.

In the proof we must show that

- 1 every output point is in the query range $[x : x']$; and
- 2 any $p \in [x : x']$ is reported.

Why does 1DRANGEQUERY not report too many points?

Consider the instructions of Algorithm 1DRANGEQUERY:

- 4 $v = \text{LEFTCHILD}(v_{\text{split}})$
- 5 while not ISLEAF(v) do
- 6 if $x \leq x_v$ then report

implied by the condition $x \leq x_v$ and at the left of v_{split} .

Exercise 2: Write a complete proof in your own words of the lemma.

an output sensitive algorithm

A balanced binary search tree takes $O(n)$ storage and $O(n \log(n))$ time to construct.

Algorithm 1DRANGEQUERY is output sensitive: if all points must be reported, then $\Theta(n)$ cannot be avoided.

Theorem (cost of 1DRANGEQUERY)

Let n points be stored in a balanced binary search tree and let k be the number of points in $[x : x']$. It takes $O(k + \log(n))$ operations to report all points in $[x : x']$.

A search in a balanced binary search tree costs $O(\log(n))$ operations.

Orthogonal Range Queries

1 1-D Range Searching

- range query and split node
- algorithm formulation and cost analysis

2 Kd Trees

- two dimensional range queries
- building a Kd tree

3 Spatial Search in CGAL

- package wrappers available for Python

two dimensional range queries

The input of a two rectangular range query is

- 1 Let P be a set of n points in the plane.

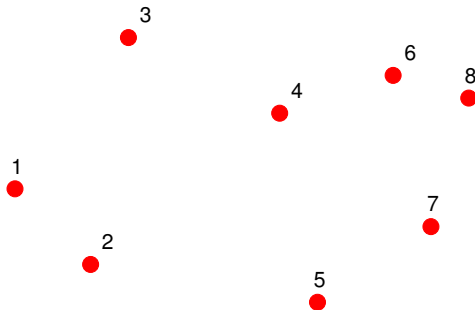
Assume: no two points have the same x -coordinate, and
no two points have the same y -coordinate.

- 2 A query rectangle $[x : x'] \times [y : y']$.

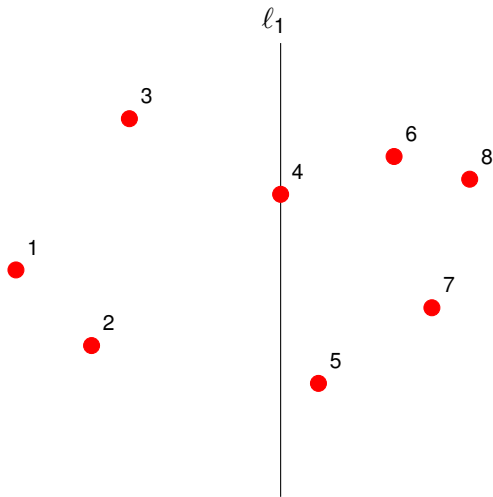
The output is $\{ (p_x, p_y) \mid x \leq p_x \leq x', y \leq p_y \leq y' \}$.

How to generalize the binary search tree for 2D range queries?

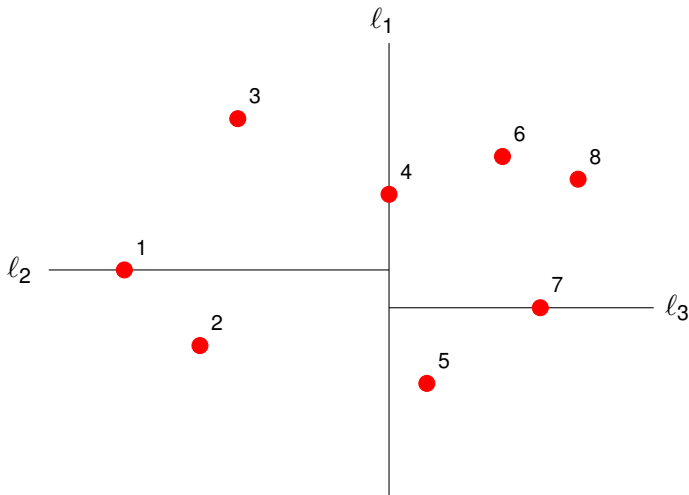
an example



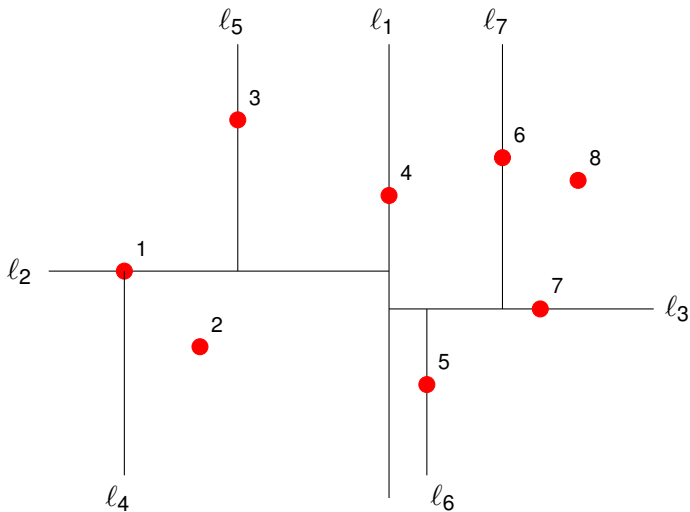
a vertical split



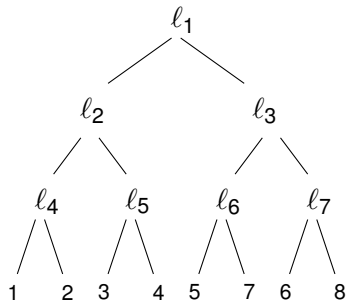
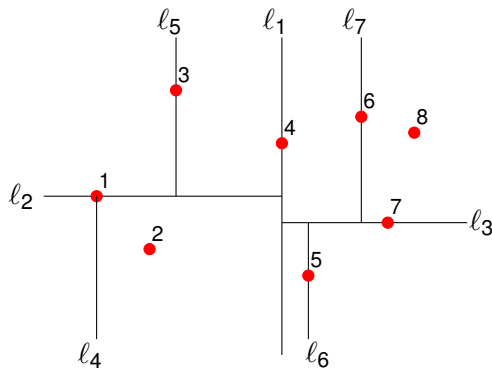
split the halves horizontally



split each quarter vertically



a two dimensional Kd tree



Left of a horizontal line is below, and right is above.
Originally, the K in Kd meant the dimension, now Kd tree is a name.

Exercise 3: For the example above, build the Kd tree step by step.

Orthogonal Range Queries

1 1-D Range Searching

- range query and split node
- algorithm formulation and cost analysis

2 Kd Trees

- two dimensional range queries
- building a Kd tree

3 Spatial Search in CGAL

- package wrappers available for Python

building a Kd tree

Algorithm BUILDKDTREE(P , depth)

Input: P is a set of points, depth is the current depth.

Output: a Kd tree stores P .

- 1 If $\#P = 1$ then
- 2 return NODE(p_1)
- else
- 3 if depth mod 2 = 0 then
- 4 ℓ = vertical line through median x of P
- 5 $(P_1, P_2) = \text{SPLIT}(P, \ell)$, P_1 is left, P_2 is right
- else
- 6 ℓ = horizontal line through median y of P
- 7 $(P_1, P_2) = \text{SPLIT}(P, \ell)$, P_1 is below, P_2 is above

BUILDKD TREE continued with recursive calls

P_1 is the left or the below part of P , ℓ is vertical, or
 P_2 is the right or the above part of P , ℓ is horizontal, and
`depth` is the current depth of the tree:

- 8 $v_{\text{left}} = \text{BUILDKD TREE}(P_1, \text{depth} + 1)$
- 9 $v_{\text{right}} = \text{BUILDKD TREE}(P_2, \text{depth} + 1)$
- 10 **return** $\text{NODE}(\ell, v_{\text{left}}, v_{\text{right}})$

Exercise 4: Review the data structures algorithms to insert and delete elements from a binary search tree. Consider the problem of inserting and deleting points in a Kd tree. Are the data structures algorithms directly applicable or easily adjustable to inserting and deleting points in a Kd tree? This exercise is related to Exercise 5.2 in the textbook.

cost of BUILDKDTREE

If P is sorted on x -coordinate and y -coordinate, then

- finding the median is $O(n)$, and
- the cost of the split is $O(n)$.

The running time of BUILDKDTREE has the recurrence

$$T(n) = \begin{cases} O(1) & \text{if } n = 1, \\ O(n) + 2T(\lceil n/2 \rceil) & \text{if } n > 1. \end{cases}$$

This recurrence has $O(n \log(n))$ as solution.

A Kd tree for n points uses $O(n)$ storage.

Orthogonal Range Queries

1 1-D Range Searching

- range query and split node
- algorithm formulation and cost analysis

2 Kd Trees

- two dimensional range queries
- building a Kd tree

3 Spatial Search in CGAL

- package wrappers available for Python

Spatial Search in CGAL

At `https://doc.cgal.org`, starting at the Package Overview, navigate to the Spatial Searching and Sorting section.

Looking at the User Manual of **dD Spatial Searching**: the search tree used in the queries is a Kd tree.

From the wiki page at github of `cgal-swig-bindings`: wrappers are provided for dD Spatial Searching (2d and 3d).

The examples of the `cgal-swig-bindings` for Python contains `test_sp.py`, which imports from `CGAL_spatial_searching`.

summary and exercises

We covered sections 5.1 and half of 5.2 in the textbook.

Consider the following activities, listed below.

- 1 Write the solutions to exercises 1, 2, 3, and 4.
- 2 Consult the CGAL documentation and example code for working with Kd trees.
- 3 Consider the exercises 1, 2, 3 in the textbook.